Lab 2: Setting Up the Database Context for a Retail Store

Scenario:

The retail store wants to store product and category data in SQL Server.

Objective:

Configure DbContext and connect to SQL Server.

Source code

Models/Category.cs

```
using System.Collections.Generic;
namespace RetailInventory.Models  {
   public class Categor  {
      public int Id { get; set; }
      public string Name { get; set; }
      public List<Product> Products { get; set; } = new List<Product>();
   }
}
```

Models/Product.cs

```
namespace RetailInventory.Models
{
   public class Product
   {
      public int Id { get; set; }
      public string Name { get; set; }
      public decimal Price { get; set; }
      public int CategoryId { get; set; }
      public Category Category { get; set; }
   }
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using RetailInventory.Models;
namespace RetailInventory.Data
{
   public class AppDbContext : DbContext
   {
      public DbSet<Product> Products { get; set; }
      public DbSet<Category> Categories { get; set; }
      protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
      {
         optionsBuilder.UseSqlServer(

@"Server=(localdb)\MSSQLLocalDB;Database=RetailStoreDb;Trusted_Connection=True;T
rustServerCertificate=True;");
      }
   }
}
```

Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using RetailInventory.Models;
namespace RetailInventory.Data
{
   public class AppDbContext : DbContext
   {
      public DbSet<Product> Products { get; set; }
      public DbSet<Category> Categories { get; set; }

      protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
```

```
        {
            IConfigurationRoot configuration = new ConfigurationBuilder()
                .SetBasePath(Directory.GetCurrentDirectory())
                .AddJsonFile("appsettings.json")
                .Build();
            string connectionString = configuration.GetConnectionString("DefaultConnection");
            optionsBuilder.UseSqlServer(connectionString);
        }
    }
}
```

Program.cs

```
using System;
using System.Linq;
using RetailInventory.Data;
using RetailInventory.Models;

namespace RetailInventor  {
    class Program {
        static void Main(string[] args)   {
            using (var context = new AppDbContext())   {
                bool created = context.Database.EnsureCreated();
                Console.WriteLine(created
                    ? "Database and tables created successfully."
                    : "Database already exists — connection confirmed.");
                if (!context.Categories.Any()) {
                    var category = new Category { Name = "Beverages" };
                    context.Categories.Add(category);
                    context.SaveChanges();
                    context.Products.Add(new Product
                    {
```

```
            Name = "Mineral Water 1L",
            Price = 25.00m,
            CategoryId = category.Id
        });
        context.SaveChanges();
        Console.WriteLine("Sample category and product inserted.");
    }
    Console.WriteLine("\n--- Categories in DB ---");
    foreach (var c in context.Categories) {
        Console.WriteLine($"Id: {c.Id}, Name: {c.Name}"); }}
    Console.WriteLine("\n--- Products in DB ---");
    foreach (var p in context.Products)      }
        Console.WriteLine($"Id: {p.Id}, Name: {p.Name}, Price: {p.Price:C},
CategoryId: {p.CategoryId}");   }}
    Console.WriteLine("\nPress any key to exit...");
    Console.ReadKey();  }}}
```

Output

```
Database and tables created successfully.
Sample category and product inserted.

--- Categories in DB ---
Id: 1, Name: Beverages

--- Products in DB ---
Id: 1, Name: Mineral Water 1L, Price: ₹25.00, CategoryId: 1

Press any key to exit...
```

```
Database already exists — connection confirmed.

--- Categories in DB ---
Id: 1, Name: Beverages

--- Products in DB ---
Id: 1, Name: Mineral Water 1L, Price: ₹25.00, CategoryId: 1

Press any key to exit...
```